Chapter 2

Instructions: Language of the Computer

Instruction Set

- The repertoire of instructions of a computer
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets

The RISC-V Instruction Set

- Used as the example throughout the book
- Developed at UC Berkeley as open ISA
- Now managed by the RISC-V Foundation (riscv.org)
- Typical of many modern ISAs
  - See RISC-V Reference Data tear-out card
- Similar ISAs have a large share of embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, …

Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

```
add a, b, c   // a gets b + c
```

- All arithmetic operations have this form
- *Design Principle 1:* Simplicity favours regularity
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

Arithmetic Example

- C code:

```
f = (g + h) - (i + j);
```

- Compiled RISC-V code:

```
add t0, g, h   // temp t0 = g + h
add t1, i, j   // temp t1 = i + j
sub f, t0, t1  // f = t0 - t1
```

Register Operands

- Arithmetic instructions use register operands
- RISC-V has a 32 × 64-bit register file
  - Use for frequently accessed data
  - 64-bit data is called a “doubleword”
    - 32 x 64-bit general purpose registers x0 to x31
  - 32-bit data is called a “word”
- *Design Principle 2:* Smaller is faster
  - c.f. main memory: millions of locations

RISC-V Registers

- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8 – x9, x18 – x27: callee-saved registers (x8 frame pointer)
- x10 – x11: function arguments/results
- x12 – x17: function arguments

Register Operand Example

- C code:

```
f = (g + h) - (i + j);
```

  - f, …, j in x19, x20, …, x23

- Compiled RISC-V code:

```
add x5, x20, x21
add x6, x22, x23
sub x19, x5, x6
```

No-Op

- A No-op is an instruction that does nothing...
  - Why?
    You may need to replace code later: No-ops can fill space, align data, and perform other options
- By ***convention*** RISC-V has a specific no-op instruction...
  - **add x0 x0 x0**
- Why?
  - Writes to x0 are always ignored...
    RISC-V uses that a lot as we will see in the jump-and-link operations
  - Making a "standard" no-op improves the disassembler and can potentially improve the processor
    - Special case the particular conventional no-op.

Memory Operands

- Main memory used for composite data
  - Arrays, structures, dynamic data
- To apply arithmetic operations
  - Load values from memory into registers
  - Store result from register to memory
- Memory is byte addressed
  - Each address identifies an 8-bit byte
- RISC-V is Little Endian
  - Least-significant byte at least address of a word
  - *c.f.* Big Endian: most-significant byte at least address
- RISC-V does not require words to be aligned in memory
  - Unlike some other ISAs

Memory Operand Example

- C code:

```
A[12] = h + A[8];
```

  - h in x21, base address of A in x22

- Compiled RISC-V code:
  - Index 8 requires offset of 64
    - 8 bytes per doubleword

```
ld      x9, 64(x22)
add     x9, x21, x9
sd      x9, 96(x22)
```

Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important!

Immediate Operands

- Constant data specified in an instruction

```
addi x22, x22, 4
```

- Make the common case fast
  - Small constants are common
  - Immediate operand avoids a load instruction

The Constant Zero

- RISC-V register x0 is the constant 0
  - Cannot be overwritten
- Useful for common operations
  - E.g., negate a value in a register by sub zero by the value
  - `sub  x9, x0, x8`

Unsigned Binary Integers

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: 0 to $+2^n - 1$
- Example
  - $0000\ 0000 \ldots 0000\ 1011_2$
    $= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$
    $= 0 + \ldots + 8 + 0 + 2 + 1 = 11_{10}$
- Using 64 bits: 0 to +18,446,774,073,709,551,615

2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Example
  - $1111\ 1111\ \ldots\ 1111\ 1100_2$
    $= -1\times2^{31} + 1\times2^{30} + \ldots + 1\times2^2 + 0\times2^1 + 0\times2^0$
    $= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$
- Using 64 bits: −9,223,372,036,854,775,808
  to 9,223,372,036,854,775,807

2s-Complement Signed Integers

- Bit 63 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- $-(-2^{n-1})$ can't be represented
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
  - 0: 0000 0000 … 0000
  - –1: 1111 1111 … 1111
  - Most-negative: 1000 0000 … 0000
  - Most-positive: 0111 1111 … 1111

Signed Negation

- Complement and add 1
  - Complement means 1 ⟶ 0, 0 ⟶ 1

$$x + \bar{x} = 1111...111_2 = -1$$

$$\bar{x} + 1 = -x$$

- Example: negate +2
  - +2 = 0000 0000 … 0010_{two}
  - –2 = 1111 1111 … 1101_{two} + 1
    = 1111 1111 … 1110_{two}

Sign Extension

- Representing a number using more bits
  - Preserve the numeric value
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - –2: 1111 1110 => 1111 1111 1111 1110

- In RISC-V instruction set
  - lb: sign-extend loaded byte
  - lbu: zero-extend loaded byte

Exercise 1

```
addi x11, x0, 0x3f5
sw x11, 0(x5)
lb x12, 1(x5)
```

What's the value in x12?

A. 0x5

B. 0xf

C. 0x3

D. 0xffffffff

Exercise 2

```
addi x11, x0, 0x80f5
sw x11, 0(x5)
lb x12, 1(x5)
```

What's the value in x12?

A. 0x80

B. 0xf5

C. 0x0

D. 0xffffff80

Representing Instructions

- Instructions are encoded in binary
  - Called machine code
- RISC-V instructions
  - Encoded as 32-bit instruction words
  - Small number of formats encoding operation code (opcode), register numbers, …
  - Regularity!

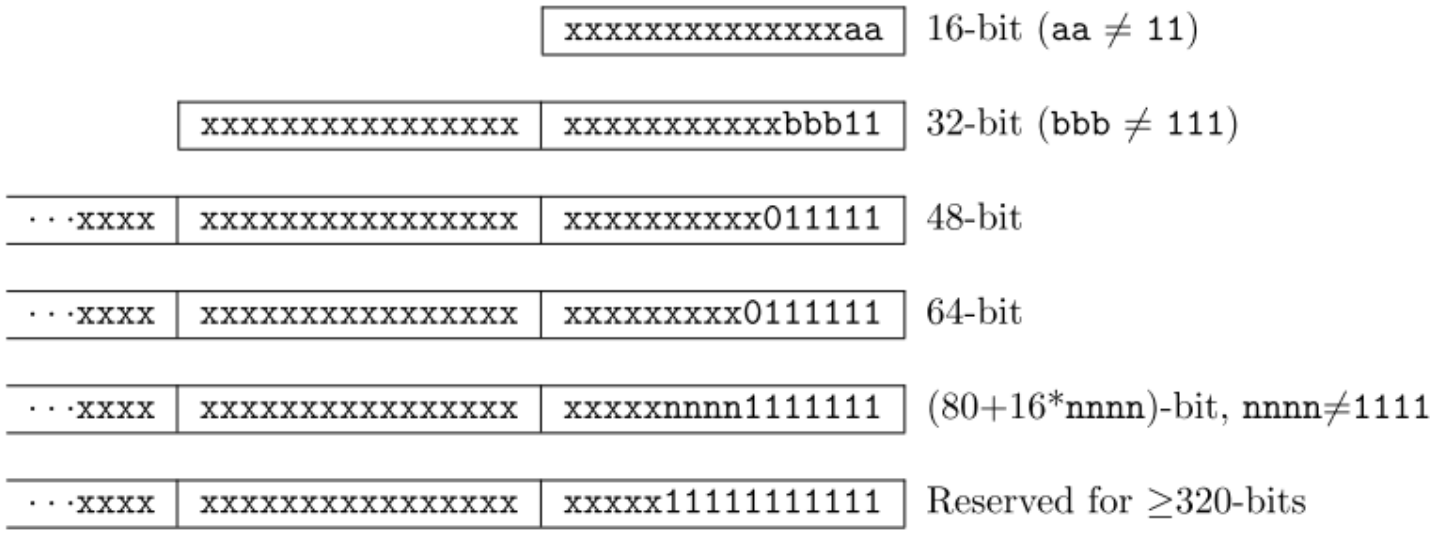
Hexadecimal

- Base 16
  - Compact representation of bit strings
  - 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

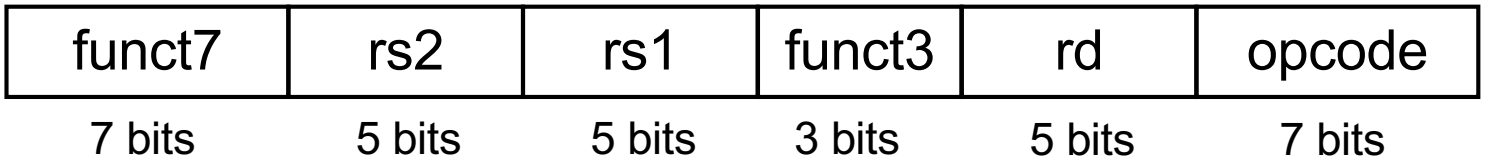
- Example: eca8 6420
  - 1110 1100 1010 1000 0110 0100 0010 0000

RISC-V Instruction Encoding



- Base instruction set (RV32 and RV64) always has fixed 32-bit instructions lowest two bits = 11_2
- All branches and jumps have targets at 16-bit granularity (even in base ISA where all instructions are fixed 32 bits)
  - Still will cause a fault if fetching a 32-bit instruction

RISC-V R-format Instructions



- Instruction fields
  - opcode: operation code
  - rd: destination register number
  - funct3: 3-bit function code (additional opcode)
  - rs1: the first source register number
  - rs2: the second source register number
  - funct7: 7-bit function code (additional opcode)

R-format Example

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

```
add x9,x20,x21
```

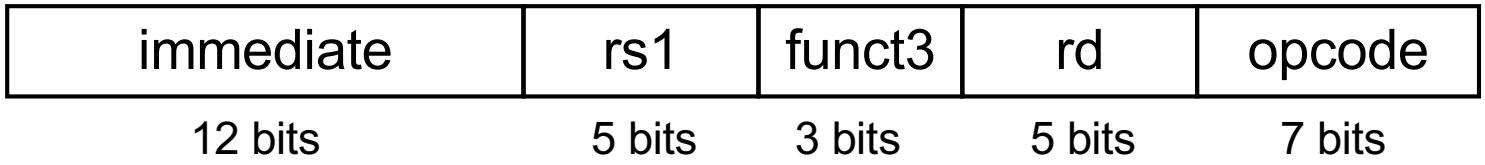
0	21	20	0	9	51

0000000	10101	10100	000	01001	0110011

$0000\ 0001\ 0101\ 1010\ 0000\ 0100\ 1011\ 0011_{two} =$

$015A04B3_{16}$

RISC-V I-format Instructions

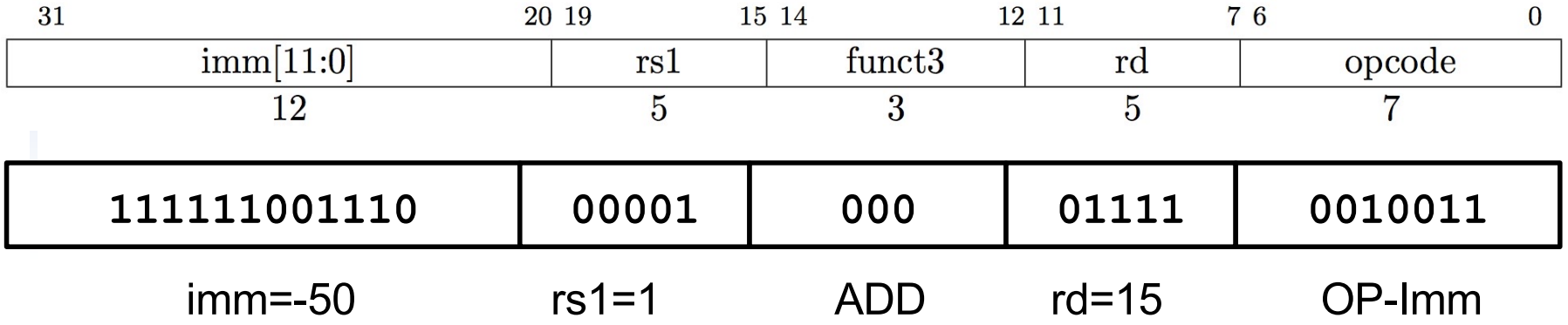


- Immediate arithmetic and load instructions
  - rs1: source or base address register number
  - immediate: constant operand, or offset added to base address
    - 2s-complement, sign extended
- *Design Principle 3:* Good design demands good compromises
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

RISC-V I-Format Instruction

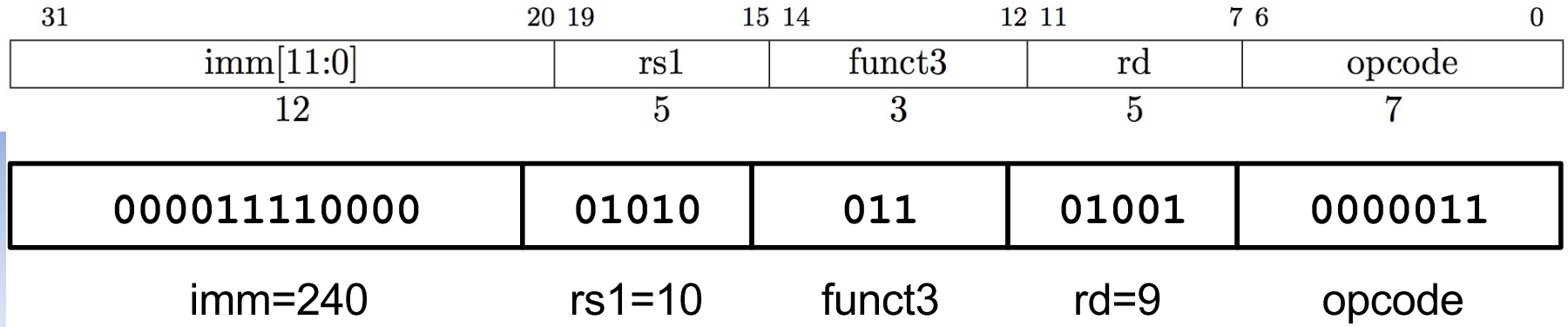
RISC-V Assembly Instruction:

```
addi  x15,x1,-50
```



- imm[11:0] can hold values in range [-2048_{ten} , $+2047_{ten}$]
- Immediate is always sign-extended to 32-bits before use in an arithmetic operation

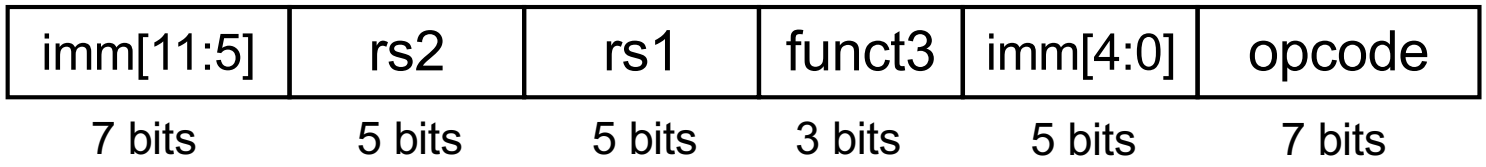
Load Instruction are also I-Type



ld x9, 240(x10)

- The 12-bit signed immediate is added to the base address in register **rs1** to form the memory address
  - This is very similar to the add-immediate operation but used to create address not to create final result
- The value loaded from memory is stored in register **rd**

RISC-V S-format Instructions

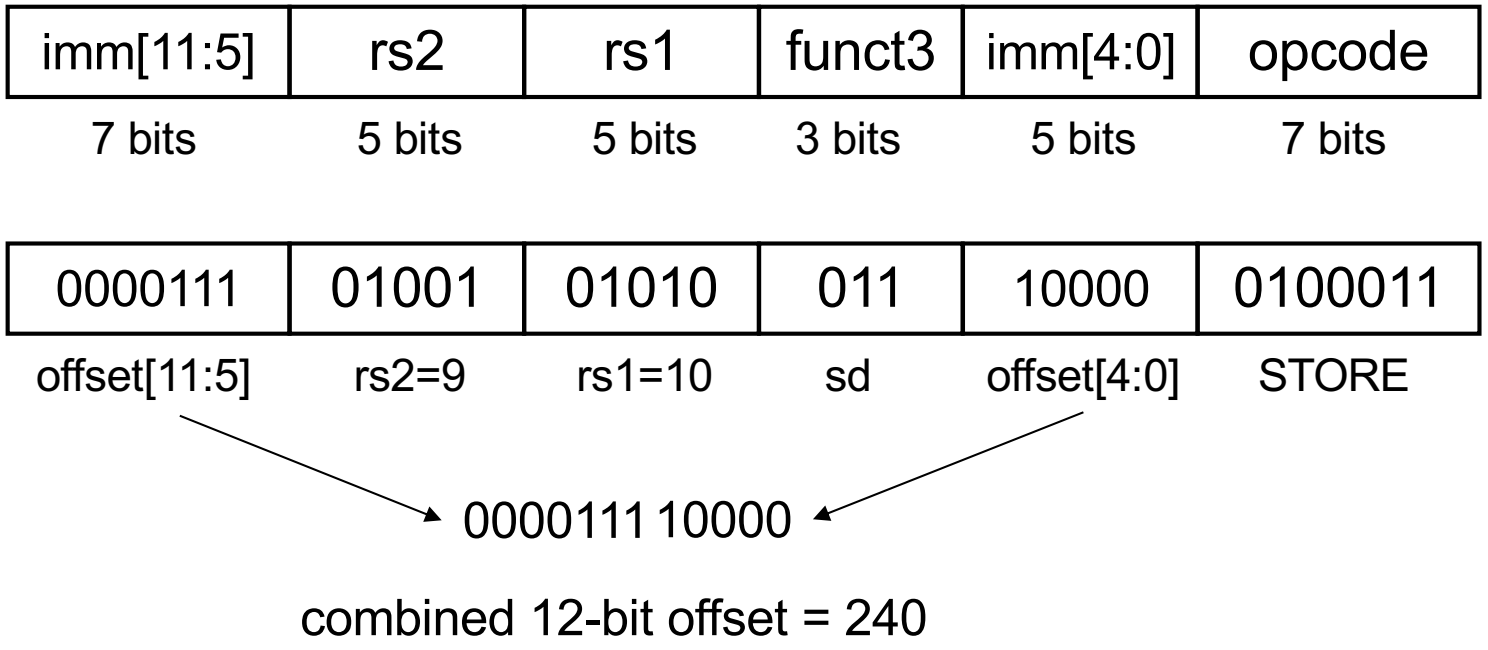


- Different immediate format for store instructions
  - rs1: base address register number
  - rs2: source operand register number
  - immediate: offset added to base address
    - Split so that rs1 and rs2 fields always in the same place

S-Format Example

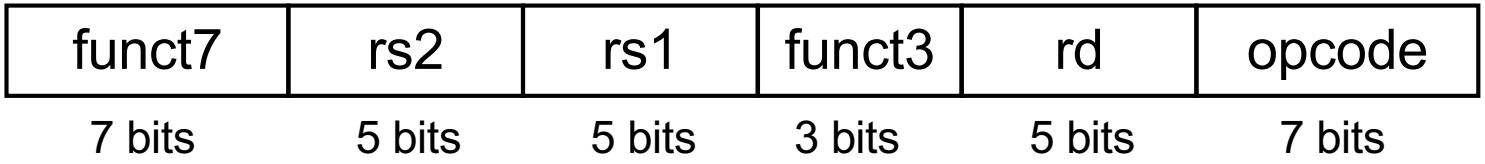
- RISC-V Assembly Instruction:

  sd x9, 240(x10)

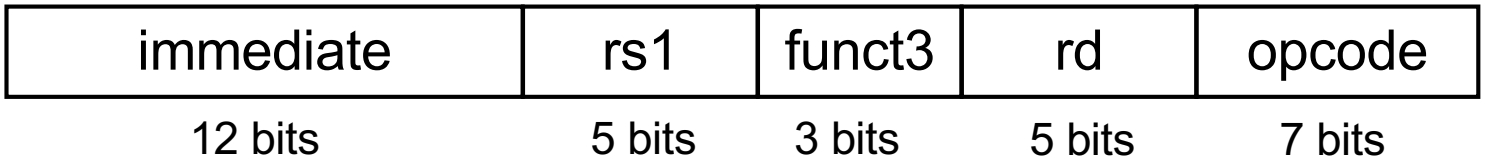


RISC-V Instruction Formats

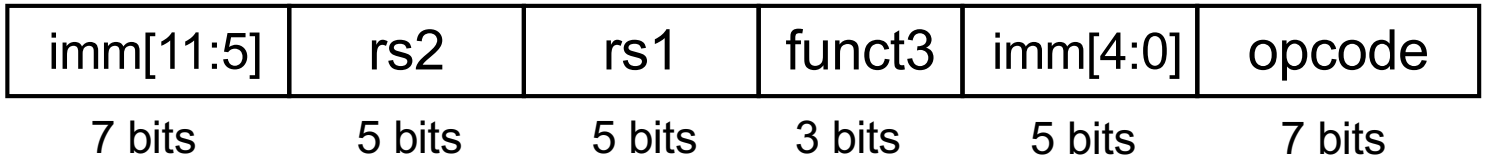
- R-format Instructions



- I-format Instructions



- S-format Instructions

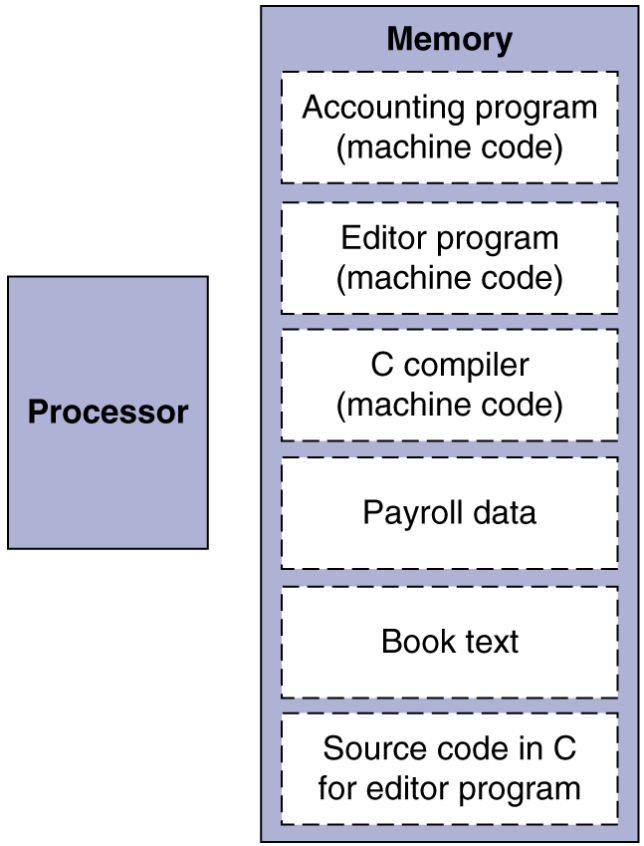


Keeping Registers always in the Same Place...

- The critical path for ***all operations*** includes fetching values from the registers
- By always placing the read sources in the same place, the register file can read without hesitation
  - If the data ends up being unnecessary (e.g. I-Type), it can be ignored
- Other RISCs have had slightly different encodings
  - Necessitating the logic to look at the instruction to determine which registers to read things work better
- Example of one of the (many) little tweaks done in RISC-V to make

Stored Program Computers

The BIG Picture



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

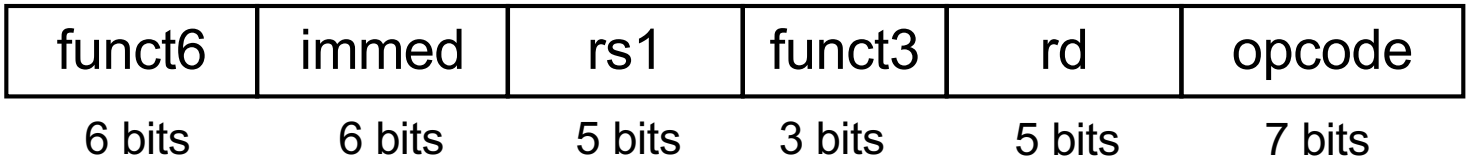
Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	RISC-V
Shift left	<<	<<	`slli`
Shift right	>>	>>>	`srli`
Bit-by-bit AND	&	&	`and, andi`
Bit-by-bit OR	|	|	`or, ori`
Bit-by-bit XOR	^	^	`xor, xori`
Bit-by-bit NOT	~	~	

- Useful for extracting and inserting groups of bits in a word

Shift Operations



- immed: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - `slli` by *i* bits multiplies by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - `srli` by *i* bits divides by 2^i (unsigned only)

Shift Operations

- Shift right arithmetic
  - `srai` moves n bits to the right (insert high-order sign bit into empty bits
  - For example, if register x10 contained 1111 1111 1111 1111 1111 1111 1110 0111_{two}= -25_{ten}
  - If execute sra x10, x10, 4, result is: 1111 1111 1111 1111 1111 1111 1111 1110_{two}= -2_{ten}

    Weaver
  - Unfortunately, this is NOT same as dividing by 2^n
    - Fails for odd negative numbers
    - C arithmetic semantics is that division should round towards 0

C90/99/11标准

6.3.7 Bitwise shift operators

Syntax

shift-expression
additive-expression
shift-expression **<<** *additive-expression*
shift-expression **>>** *additive-expression*

Constraints

Each of the operands shall have integral type.

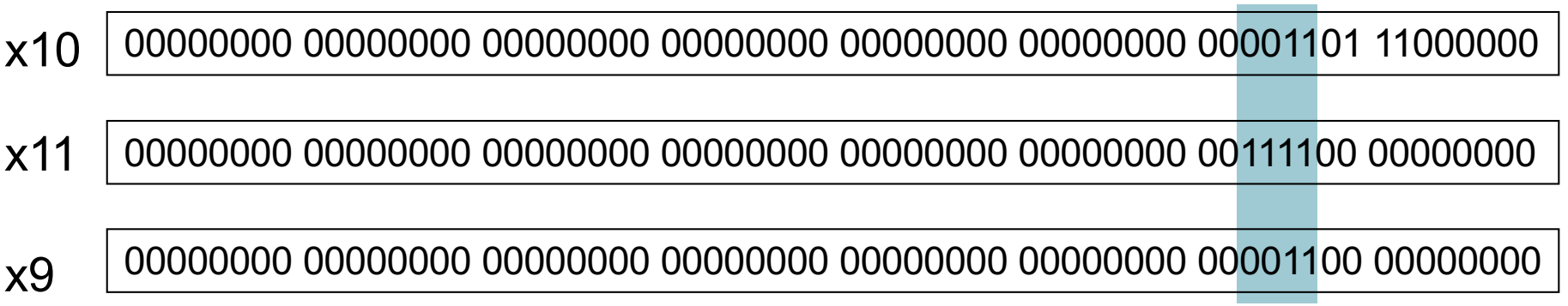
Semantics

The integral promotions are performed on each of the operands. The type of the result is that of the promoted left operand. If the value of the right operand is negative or is greater than or equal to the width in bits of the promoted left operand, the behavior is undefined.

AND Operations

- Useful to mask bits in a word
  - Select some bits, clear others to 0

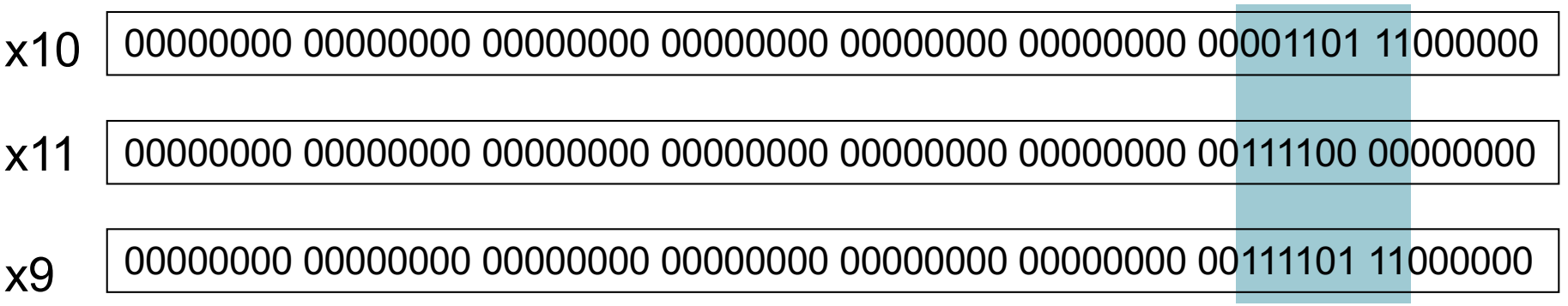
```
and x9,x10,x11
```



OR Operations

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged

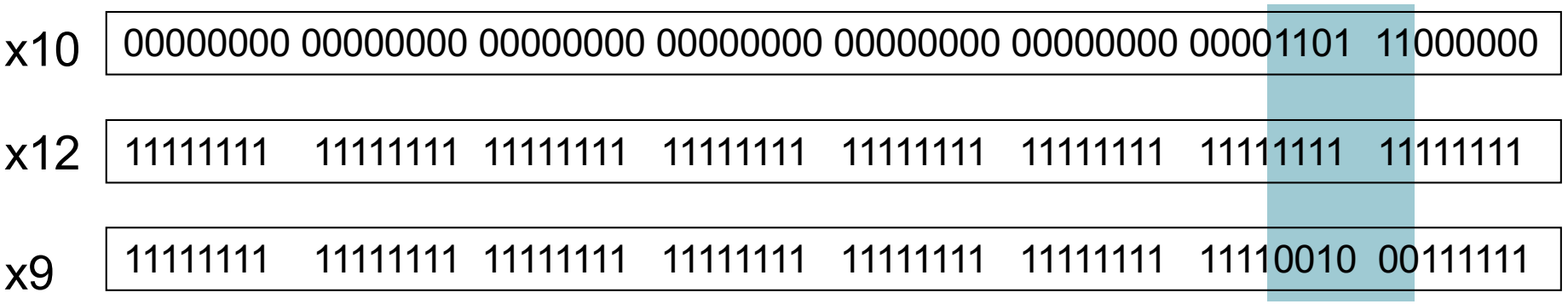
```
or x9,x10,x11
```



XOR Operations

- Differencing operation
  - Set some bits to 1, leave others unchanged

```
xor x9,x10,x12  // NOT operation
```



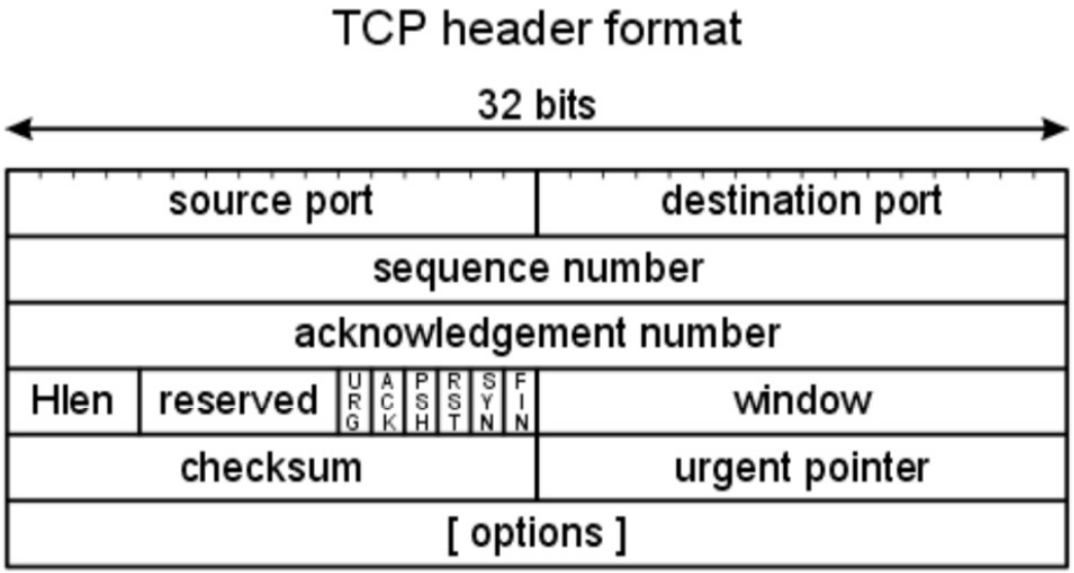
Why Shifts and Logical Operations?

- Often have to pack/unpack fields
- e.g, in C:

```
int *packet;
packet[0] = sport << 16 | dport;
```

- Becomes (packet in x1, sport in x2, dport in x3)

```
slli x4, x2, 16
or x4, x4, x3
sw x4, 0(x1)
```



Conditional Operations

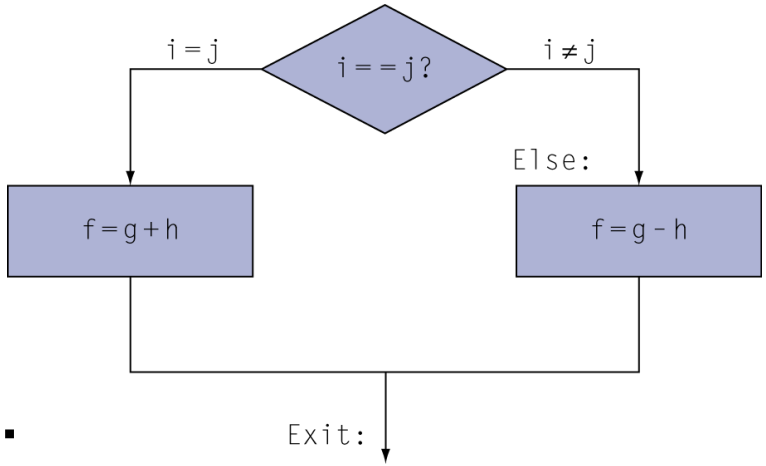
- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- `beq rs1, rs2, L1`
  - if (rs1 == rs2) branch to instruction labeled L1
- `bne rs1, rs2, L1`
  - if (rs1 != rs2) branch to instruction labeled L1

Compiling If Statements

- C code:

```
if (i==j) f = g+h;
else f = g-h;
```

  - f, g, … in x19, x20, …



- Compiled RISC-V code:

```
      bne x22, x23, Else
      add x19, x20, x21
      beq x0,x0,Exit // unconditional
Else: sub x19, x20, x21
Exit: …
```

Assembler calculates addresses

Compiling Loop Statements

- C code:

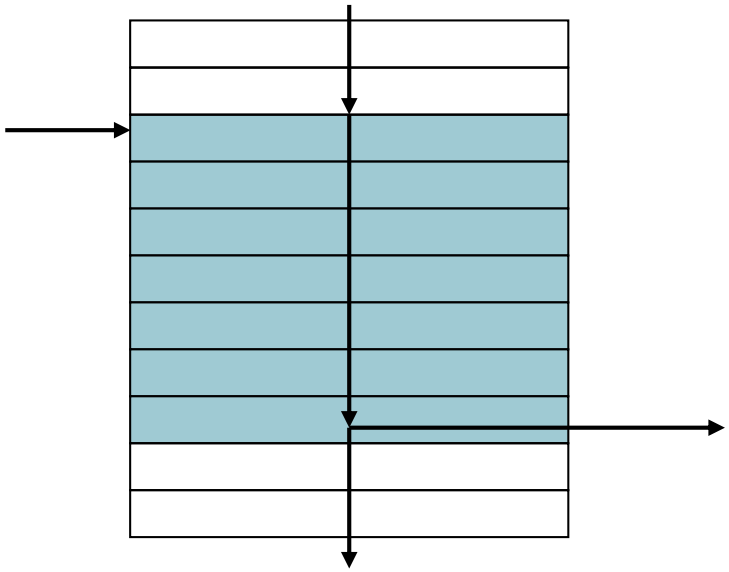
```
while (save[i] == k) i += 1;
```

  - i in x22, k in x24, address of save in x25
- Compiled RISC-V code:

```
Loop: slli x10, x22, 3
      add  x10, x10, x25
      ld   x9, 0(x10)
      bne  x9, x24, Exit
      addi x22, x22, 1
      beq  x0, x0, Loop
Exit: …
```

Basic Blocks

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)



- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks

More Conditional Operations

- `blt rs1, rs2, L1`
  - if (rs1 < rs2) branch to instruction labeled L1
- `bge rs1, rs2, L1`
  - if (rs1 >= rs2) branch to instruction labeled L1
- Example
  - if (a > b) a += 1;
  - a in x22, b in x23

```
    bge  x23, x22, Exit      // branch if b >= a
    addi x22, x22, 1
Exit:
```

Signed vs. Unsigned

- Signed comparison: blt, bge
- Unsigned comparison: bltu, bgeu
- Example
  - x22 = 1111 1111 1111 1111 1111 1111 1111 1111
  - x23 = 0000 0000 0000 0000 0000 0000 0000 0001
  - `x22 < x23 // signed`
    - –1 < +1
  - `x22 > x23 // unsigned`
    - +4,294,967,295 > +1

Procedure Calling

- Steps required
  1. Place parameters in registers x10 to x17
  2. Transfer control to procedure
  3. Acquire storage for procedure
  4. Perform procedure's operations
  5. Place result in register for caller
  6. Return to place of call (address in x1)

The "ABI" Conventions & Mnemonic Registers

- The "Application Binary Interface" defines our 'calling convention'
  - How to call other functions
- A critical portion is "what do registers mean by convention"
  - We have 32 registers, but how are they used
- Who is responsible for saving registers?
  - ABI defines a contract: When you call another function, that function promises ***not*** to overwrite certain registers
- We also have more convenient names based on this
  - So going forward, no more x3, x6... type notation

The Calling Convention: A Contract Between Functions...

- The “Calling Convention” in the ABI is the format/usage of registers in a way between the function ***caller*** and function ***callee***, if all functions implement it, everything works out
  - It is effectively a contract between functions
- Registers are two types
  - ***caller-saved***
    - The function invoked (the callee) can do whatever it wants to them!
  - ***callee-saved***
    - The function invoked must restore them before returning (if used)

The RISC-V Registers and Convention

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	——
x1	ra	Returned address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	——
x4	tp	Thread pointer	——
x5	t0	Temporary/alternate link register	Caller
x6-7	t1-2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

RISC-V Function Call Conventions

- Registers faster than memory, so use them
- **a0–a7** (**x10-x17**): eight argument registers to pass parameters and two return values (**a0-a1**) (caller saved)
  - Any more arguments should be passed on the stack
  - Technically we could return in **a2-a7** as well, but we're mostly dealing with C and not python or golang...
- **ra**: one return address register for return to the point of origin (x1) (caller saved)
- **sp**: pointer to the bottom of the stack (callee saved)

More Conventions

- **s0-s11** Callee saved registers: Preserved across function calls
- **fp** Frame Pointer: Pointer to the top of the call frame
  - Also is **s0**, the first saved register, callee saved
  - Frame pointer can often be omitted by the compiler, but we will use it because it makes things clearer how functions are translated.
- **t0-t6** Temporaries: Caller saved

Procedure Call Instructions

- Procedure call: jump and link

  `jal x1, ProcedureLabel`

  - Address of following instruction put in x1
  - Jumps to target address (PC + immediate)
- Procedure return: jump and link register

  `jalr x0, 0(x1)`

  - Like jal, but jumps to 0 + address in x1
  - Use x0 as rd (x0 cannot be changed)
  - Can also be used for computed jumps
    - e.g., for case/switch statements

Leaf Procedure Example

- C code:

```
long long int leaf_example (
    long long int g, long long int h,
    long long int i, long long int j) {
  long long int f;
  f = (g + h) - (i + j);
  return f;
}
```

  - Arguments g, …, j in x10, …, x13
  - f in x20
  - temporaries x5, x6
  - Need to save x5, x6, x20 on stack

Where Are Old Register Values Saved to Restore Them After Function Call?

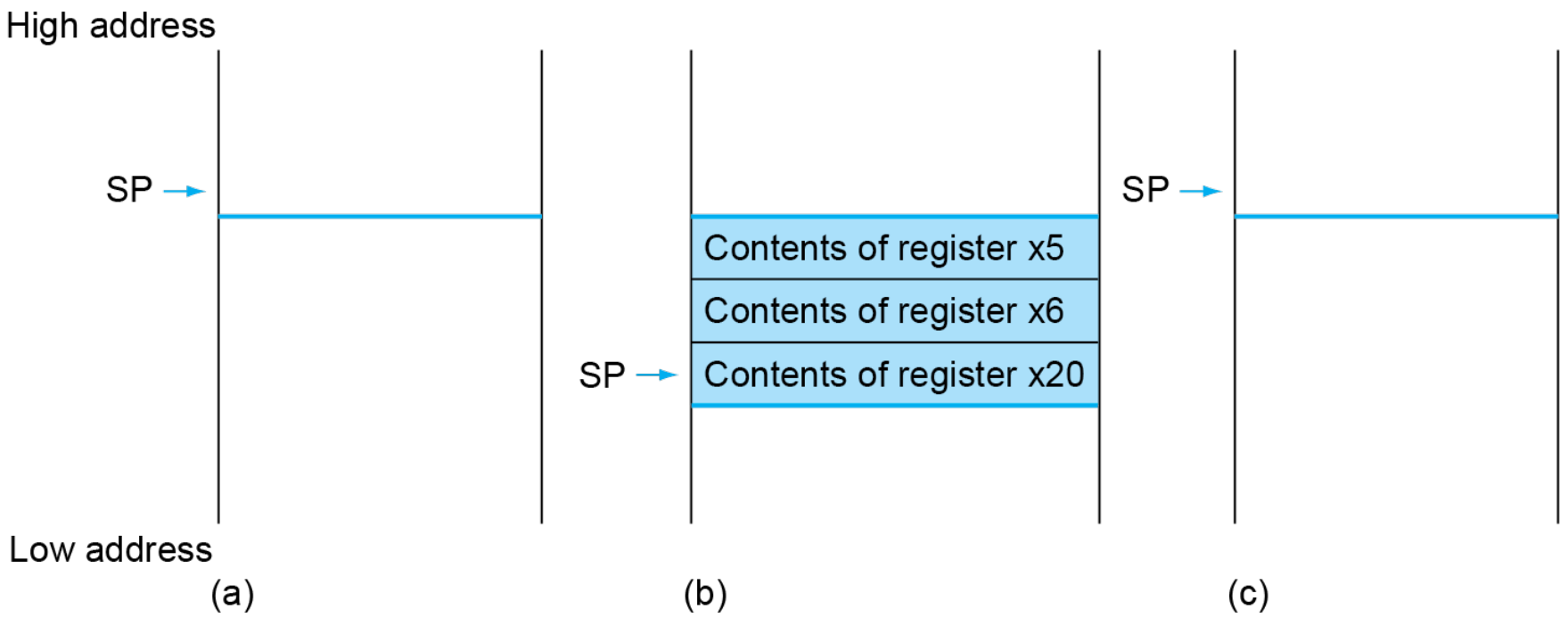
- Need a place to save old values before call function, restore them when return
- Ideal is *stack*: last-in-first-out queue (e.g., stack of plates)
  - Push: placing data onto stack
  - Pop: removing data from stack
- Stack in memory, so need register to point to it
- **sp** is the *stack pointer* in RISC-V (**x2**)
- **sp** always points to the last used place on the stack
- Convention is grow stack down from high to low addresses
  - *Push* decrements **sp**, *Pop* increments **sp**

Leaf Procedure Example

- RISC-V code:

```
leaf_example:
   addi sp,sp,-24          Save x5, x6, x20 on stack
   sd   x5,16(sp)
   sd   x6,8(sp)
   sd   x20,0(sp)
   add  x5,x10,x11         x5 = g + h
   add  x6,x12,x1          x6 = i + j
   sub  x20,x5,x6          f = x5 – x6
   addi x10,x20,0          copy f to return register
   ld   x20,0(sp)          Resore x5, x6, x20 from stack
   ld   x6,8(sp)
   ld   x5,16(sp)
   addi sp,sp,24
   jalr x0,0(x1)           Return to caller
```

Local Data on the Stack



Non-Leaf Procedures

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

Non-Leaf Procedure Example

- C code:

```
long long int fact (long long int n)
{
  if (n < 1) return 1;
  else return n * fact(n - 1);
}
```

  - Argument n in x10
  - Result in x10

Leaf Procedure Example

- RISC-V code:

```
fact:
    addi sp,sp,-16      Save return address and n on stack
    sd   x1,8(sp)
    sd   x10,0(sp)
    addi x5,x10,-1      x5 = n - 1
    bge  x5,x0,L1       if n >= 1, go to L1
    addi x10,x0,1       Else, set return value to 1
    addi sp,sp,16       Pop stack, don't bother restoring values
    jalr x0,0(x1)       Return
L1: addi x10,x10,-1     n = n - 1
    jal  x1,fact        call fact(n-1)
    addi x6,x10,0       move result of fact(n - 1) to x6
    ld   x10,0(sp)      Restore caller's n
    ld   x1,8(sp)       Restore caller's return address
    addi sp,sp,16       Pop stack
    mul  x10,x10,x6     return n * fact(n-1)
    jalr x0,0(x1)       return
```

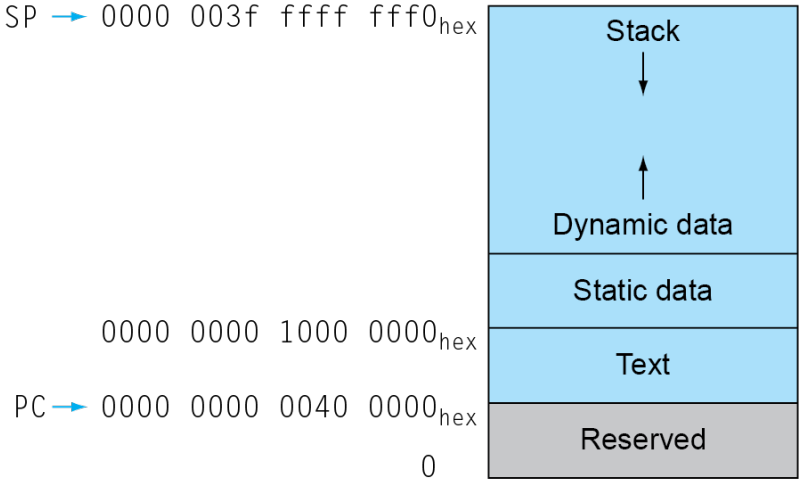
- C code:

```
long long int fact (long long int n)
{
  if (n < 1)
      return 1;
  else
      return n*fact(n - 1);
}
```

  - Argument n in x10
  - Result in x10

Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - x3 (global pointer) initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



Allocating Space on Stack

- C has two storage classes: automatic and static
  - *Automatic* variables are local to function and discarded when function exits
  - *Static* variables exist across exits from and entries to procedures
- Use stack for automatic (local) variables that aren't in registers
- *Procedure frame* or *activation record*: segment of stack with saved registers and local variables

So What Is This "Frame Pointer" Business

- As a reminder, we shove all the C local variables etc on the stack...
  - Combined with space for all the saved registers
  - This is called the “activation record” or “call frame” or “call record”
- But a naive compiler may cause the stack pointer to bounce up and down during a function call
  - Can be a lot simpler to have a compiler do a bunch of pushes and pops when it needs a bit of temporary space: more so on a CISC rather than a RISC however
- Plus not all programming languages can store all activation records on the stack:
  - The use of lambda in Scheme, Python, Go, etc requires that some call frames are allocated on the heap since variables may last beyond the function call!

So The Convention: Use s0 as a Frame Pointer (fp)

- At the start, save **s0** and then have the Frame pointer point to one below the sp when you were called...

```
addi sp sp -20      # Initially grabbing 5 words of space
sw ra sp 16         #
sw fp sp 12         # save fp/s0
addi fp sp 16       # Points to the start of this call record
```

- Now we can address local variables off the frame pointer rather than the stack pointer
  - Simplifies the compiler
    - Since it can now move the stack up and down easily
  - Simplifies the **debugger**

But Note...

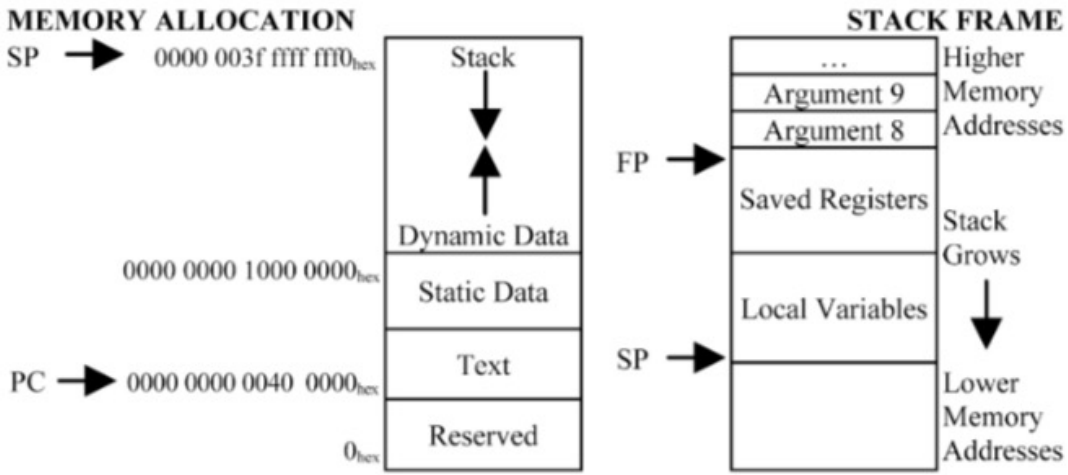
- It isn't necessary in C...
  - Most C compilers has a -f-omit-frame-pointer option on most architectures
    - It just fubars debugging a bit
- So for our hand-written assembly, we will generally ignore the frame pointer
- The calling convention says it doesn't matter if you use a frame pointer or not!
  - It is just a callee saved register, so if you use it as a frame pointer... It will be preserved just like any other saved register. But if you just use it as **s0**, that makes no difference!

The Stack Is Also For Local Variables...

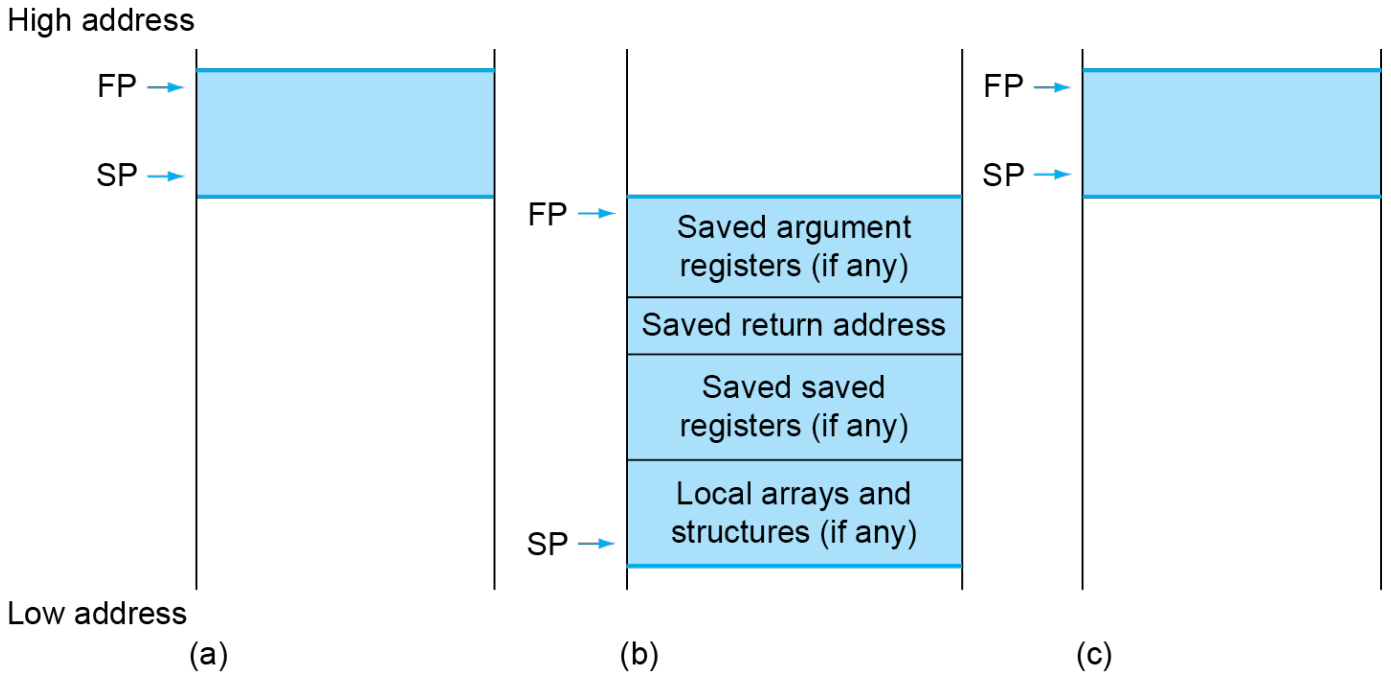
- e.g.
  char[20] foo;
- Requires enough space on the stack
  - May need padding
- So then to pass **foo** to something in **a0**...
  - **addi a0 sp *offset-for-foo-off-sp***
  - **addi a0 fp *offset-for-foo-off-fp***
    - If you are using the frame pointer...

The Stack Is Also For Arguments

- Arguments 1-8 are passed in **a0-a7**
- But what about a 9th argument or more?
  - Pass those on the stack!
- When the function is called,
  - **0(sp)** -> arg #9
  - **4(sp)** -> arg #10...



Local Data on the Stack



- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

Exercise

Which statement is FALSE?

A. RISC-V uses jal to invoke a function and jr to return from a function

B. jal saves PC+1 in ra

C. The callee can use temporary registers (t_i) without saving and restoring them

D. The caller can rely on save registers (s_i) without fear of callee changing them

Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

Byte/Halfword/Word Operations

- RISC-V byte/halfword/word load/store
  - Load byte/halfword/word: Sign extend to 64 bits in rd
    - `lb rd, offset(rs1)`
    - `lh rd, offset(rs1)`
    - `lw rd, offset(rs1)`
  - Load byte/halfword/word unsigned: Zero extend to 64 bits in rd
    - `lbu rd, offset(rs1)`
    - `lhu rd, offset(rs1)`
    - `lwu rd, offset(rs1)`
  - Store byte/halfword/word: Store rightmost 8/16/32 bits
    - `sb rs2, offset(rs1)`
    - `sh rs2, offset(rs1)`
    - `sw rs2, offset(rs1)`

String Copy Example

- C code:
  - Null-terminated string

```
void strcpy (char x[], char y[])
{ size_t i;
  i = 0;
  while ((x[i]=y[i])!='\0')
    i += 1;
}
```

String Copy Example

```
void strcpy (char x[], char y[])
{ size_t i;
  i = 0;
  while ((x[i]=y[i])!='\0')
    i += 1;
}
```

- RISC-V code:

```
strcpy:
    addi sp,sp,-8     // adjust stack for 1 double word
    sd   x19,0(sp)    // push x19
    add  x19,x0,x0    // i=0
L1: add  x5,x19,x10   // x5 = addr of y[i]
    lbu  x6,0(x5)     // x6 = y[i]
    add  x7,x19,x10   // x7 = addr of x[i]
    sb   x6,0(x7)     // x[i] = y[i]
    beq  x6,x0,L2     // if y[i] == 0 then exit
    addi x19,x19,1    // i = i + 1
    jal  x0,L1        // next iteration of loop
L2: ld   x19,0(sp)     // restore saved x19
    addi sp,sp,8      // pop 1 double word from stack
    jalr x0,0(x1)     // and return
```

Excercise

Which of the following is TRUE?

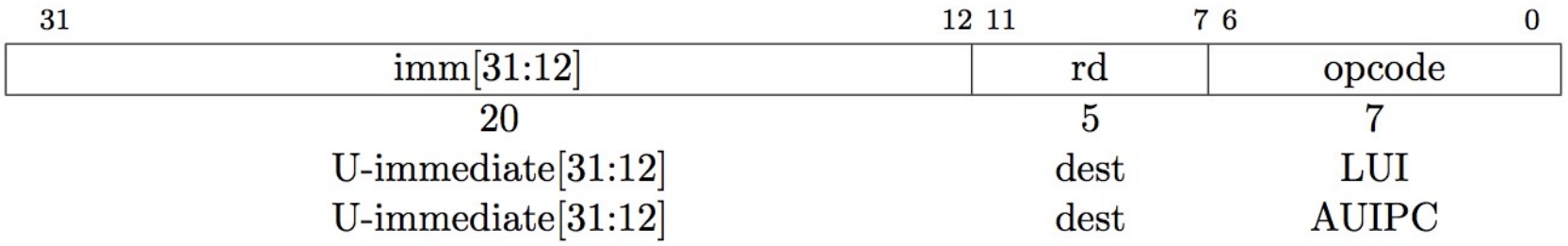
A. add x10,x11,4(x12) is valid in RV32
B. can byte address 8GB of memory with an RV32 word
C. imm must be multiple of 4 for **lw x10,imm(x10)** to be valid
D. None of the above

32-bit Constants

- Most constants are small
  - 12-bit immediate is sufficient
- For the occasional 32-bit constant

  `lui rd, constant`
  - Copies 20-bit constant to bits [31:12] of rd
  - Extends bit 31 to bits [63:32]
  - Clears bits [11:0] of rd to 0

U-Format for “Upper Immediate” instructions



- Has 20-bit immediate in upper 20 bits of 32-bit instruction word
- One destination register, rd
- Used for two instructions
  - LUI – Load Upper Immediate
  - AUIPC – Add Upper Immediate to PC

LUI to create long immediates

- LUI Copies 20-bit constant to bits [31:12] of rd, Extends bit 31 to bits [63:32], Clears bits [11:0] of rd to 0.
- Together with an ADDI to set low 12 bits, can create any 32-bit value in a register using two instructions (LUI/ADDI).

```
lui x19, 976  // 0x003D0
```

0000 0000 0000 0000	0000 0000 0000 0000	0000 0000 0011 1101 0000	0000 0000 0000

```
addi x19,x19,128  // 0x500
```

0000 0000 0000 0000	0000 0000 0000 0000	0000 0000 0011 1101 0000	0101 0000 0000

思考：如果要设置一个32位的常数0xDEADBEEF怎么办?

One Corner Case

How to set 0xDEADBEEF?

LUI x10, 0xDEADB # x10 = 0xDEADB000

ADDI x10, x10, 0xEEF# x10 = 0xDEADAEEF

ADDI 12-bit immediate is always sign-extended, if top bit is set, will subtract -1 from upper 20 bits

Solution

How to set 0xDEADBEEF?

LUI x10, 0xDEADC **# x10 = 0xDEADC000**

ADDI x10, x10, 0xEEF **# x10 = 0xDEADBEEF**

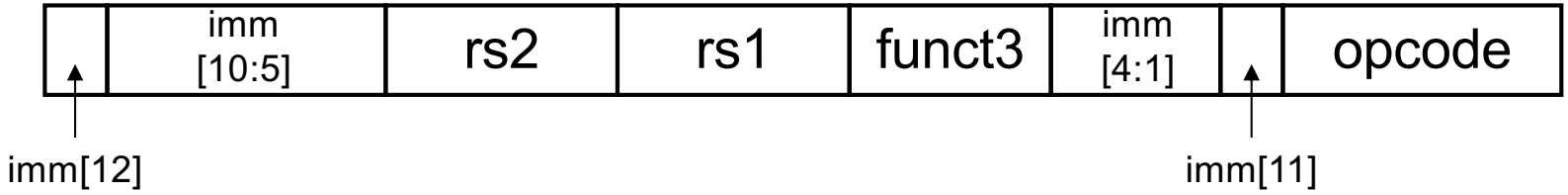
Pre-increment the value placed in upper 20 bits, if sign bit will be set on immediate in lower 12 bits.

Assembler pseudo-op handles all of this:

li x10, 0xDEADBEEF # Creates two instructions

Branch Addressing

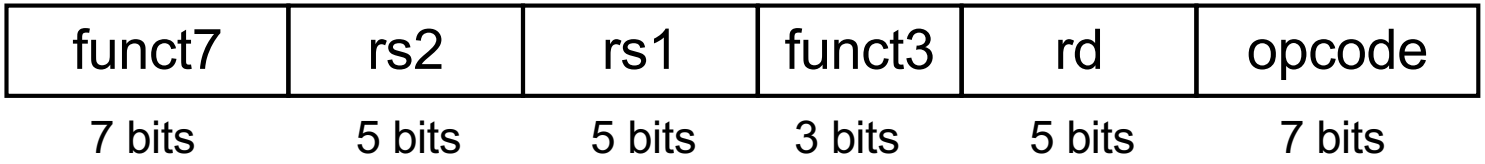
- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward
- SB format, mostly same as S-Format:



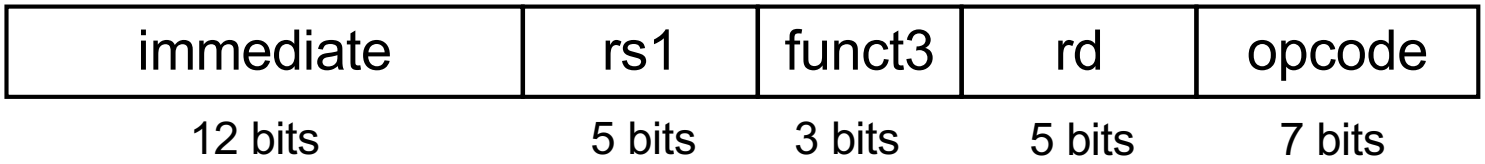
- PC-relative addressing
  - Target address = PC + immediate × 2

RISC-V Instruction Formats

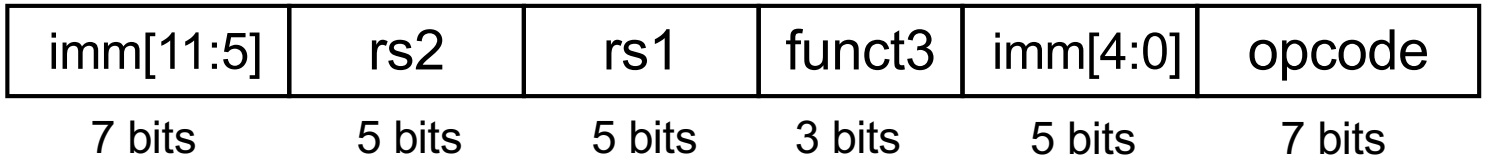
- R-format Instructions



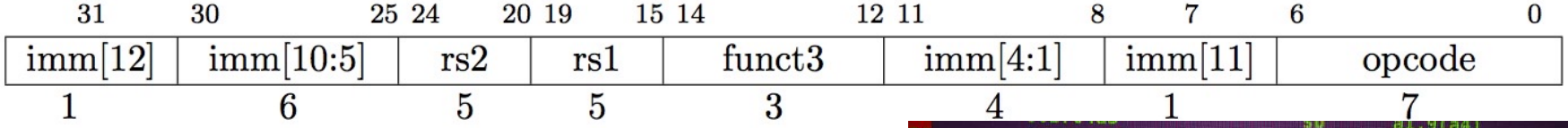
- I-format Instructions



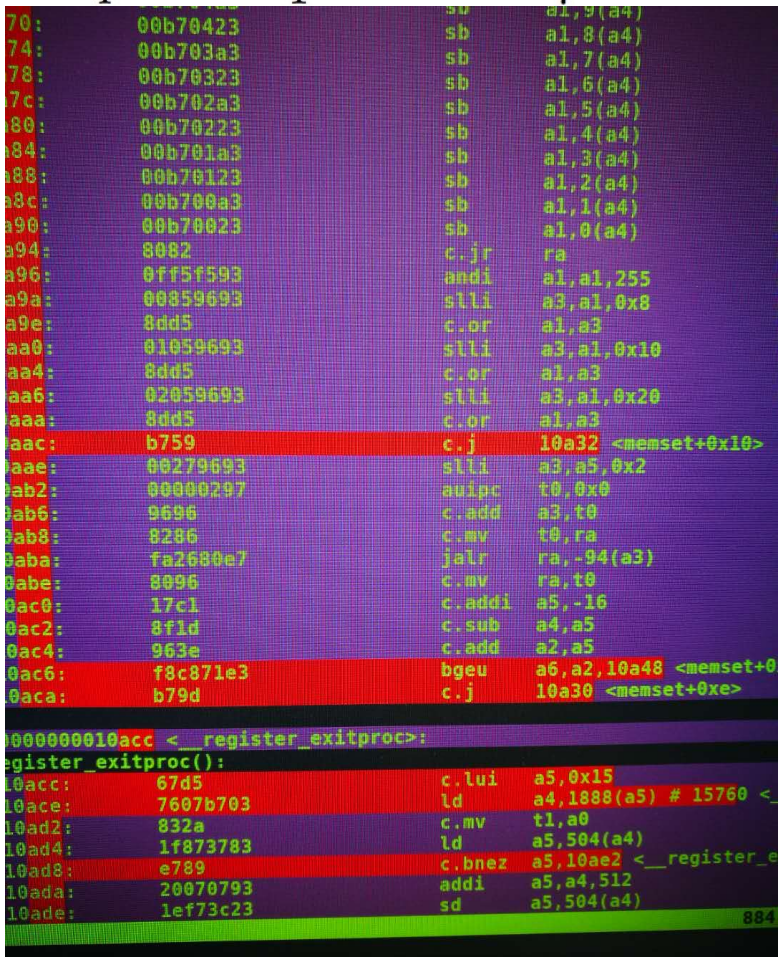
- S-format Instructions



RISC-V SB-Format for Branches



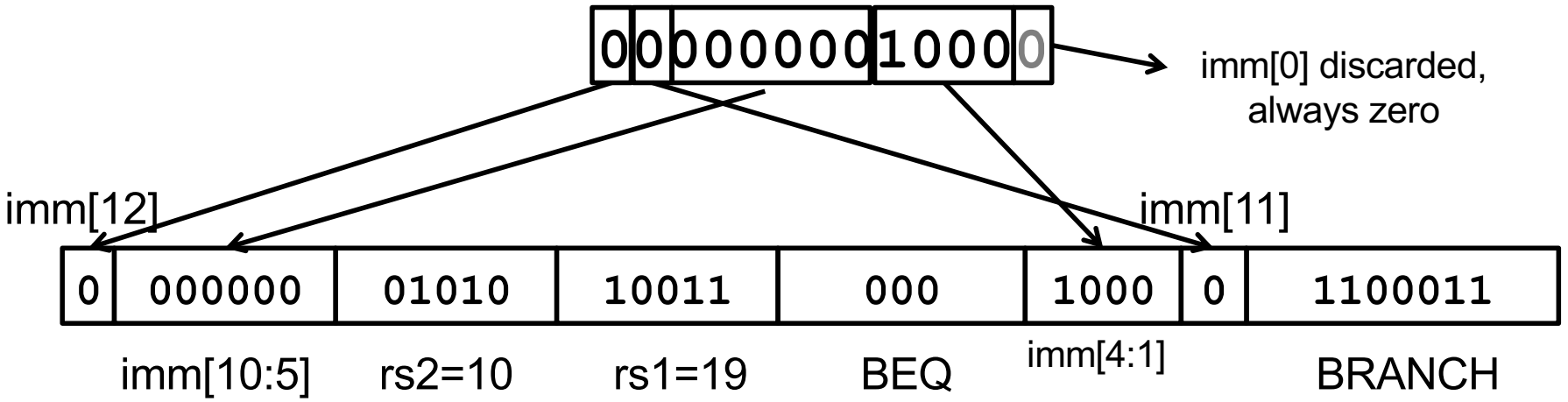
- SB-format is mostly same as S-Format, with two register sources (rs1/rs2) and a 12-bit immediate
- But now immediate represents values -4096 to +4094 in 2-byte increments
- The 12 immediate bits encode *even* 13-bit signed byte offsets (lowest bit of offset is always zero, so no need to store it)



Branch Example

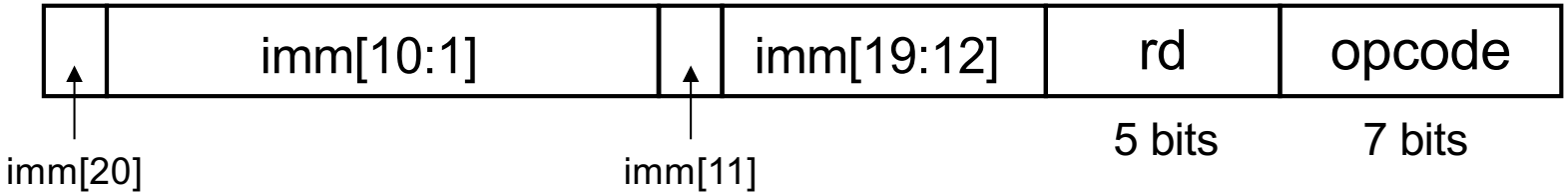
beq x19,x10, offset = 16 bytes

13-bit immediate, imm[12:0], with value 16

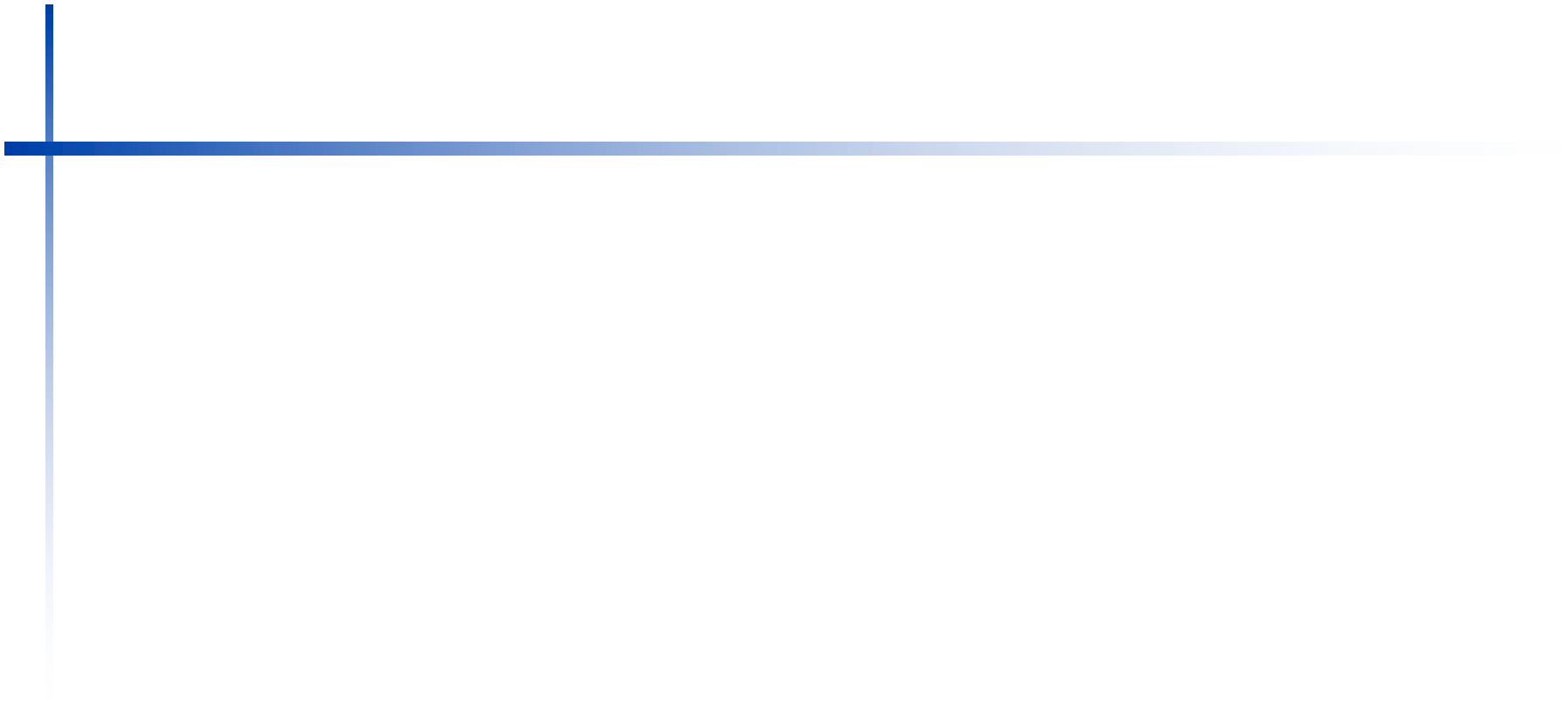


Jump Addressing

- Jump and link (`jal`) saves PC+4 in register rd (the return address)
  - Assembler ”j” jump is pseudo-instruction, uses JAL but set rd=x0 to discard return address
- target uses 20-bit immediate for larger range: Target somewhere within $\pm 2^{19}$ locations, 2 bytes apart
  - $\pm 2^{18}$ 32-bit instructions
- Immediate encoding optimized similarly to branch instruction to reduce hardware cost
- UJ format:



- For long jumps, eg, to 32-bit absolute address
  - lui: load address[31:12] to temp register
  - jalr: add address[11:0] and jump to target



Uses of JAL

j pseudo-instruction

j Label = jal x0, Label # Discard return address

Call function within 2^{18} instructions of PC

jal ra, FuncName

JALR Instruction (I-Format)

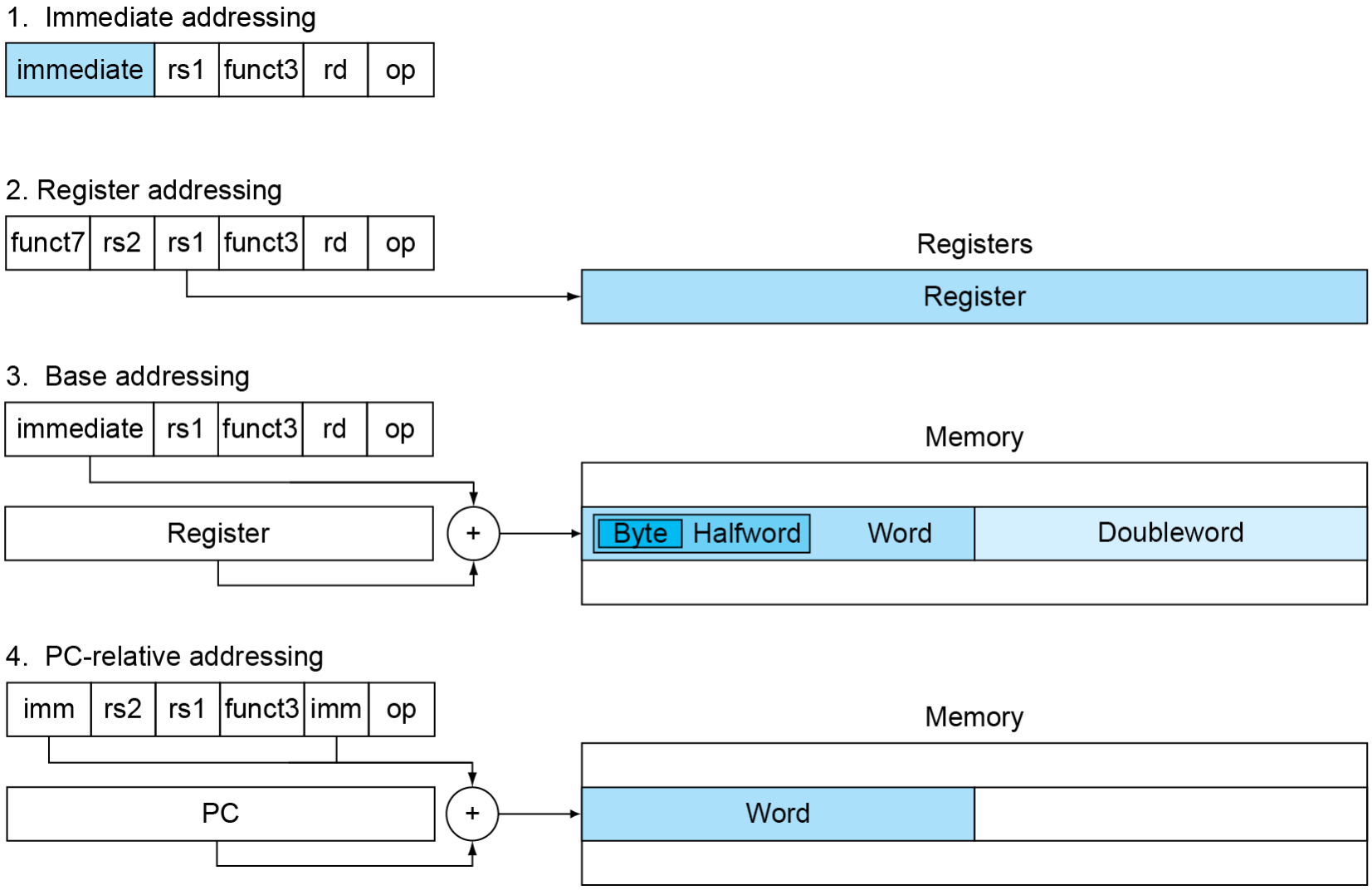
immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits
offset[11:0]	base	0	dest	JALR

- **JALR rd, rs, immediate**
  - Writes PC+4 to **rd** (return address)
  - Sets PC = **rs + immediate**
  - Uses same immediates as arithmetic and loads
    - ***no*** multiplication by 2 bytes

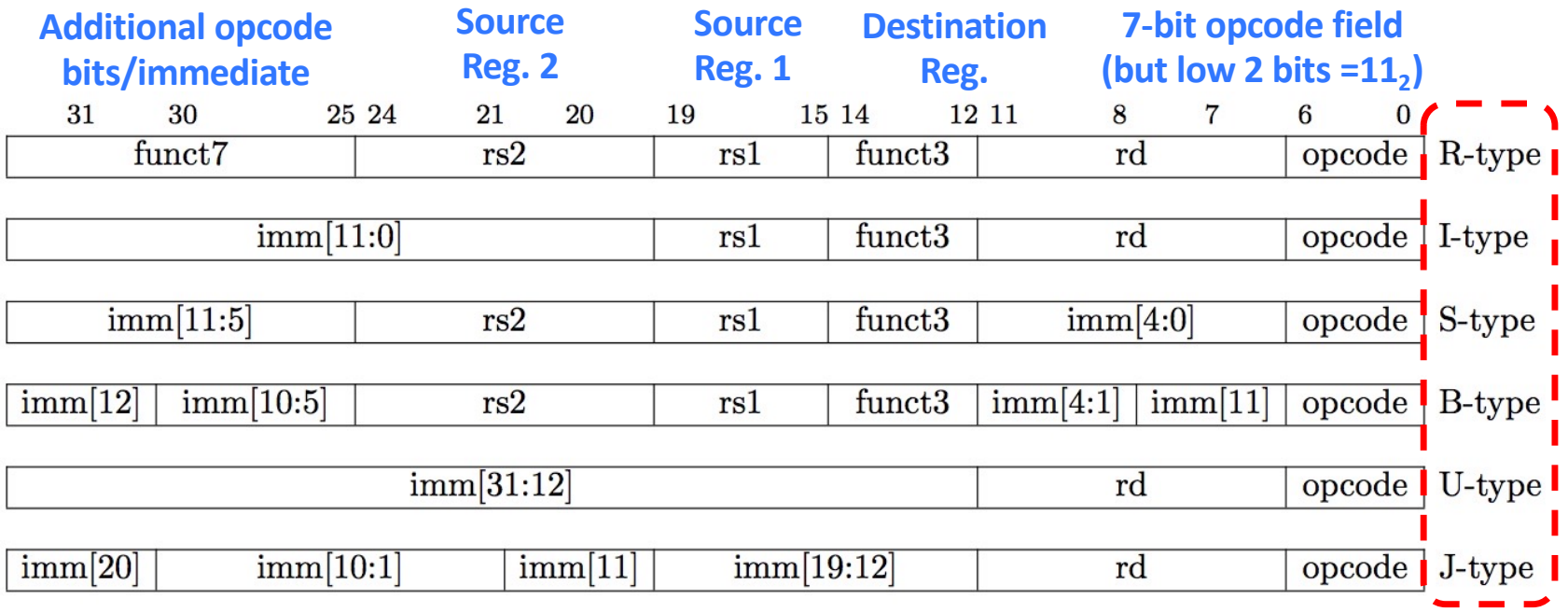
Uses of JALR

```
# ret and jr psuedo-instructions
ret = jr ra = jalr x0, ra, 0

# Call function at any 32-bit absolute address
lui x1, <hi20bits>
jalr ra, x1, <lo12bits>

# Jump PC-relative with 32-bit offset
auipc x1, <hi20bits>      # Adds upper immediate value to
                          # and places result in x1
jalr x0, x1, <lo12bits>   # Same sign extension trick needed
                          # as LUI
```

RISC-V Addressing Summary



Summary of RISC-V Instruction Formats



- Aligned on a four-byte boundary in memory
- Sign bit of immediates always on bit 31 of instruction.
- Register fields never move
- Opcode fields is on the right

Synchronization

- Two processors sharing an area of memory
  - P1 writes, then P2 reads
  - Data race if P1 and P2 don’t synchronize
    - Result depends of order of accesses
- Hardware support required
  - Atomic read/write memory operation
  - No other access to the location allowed between the read and write
- Could be a single instruction
  - E.g., atomic swap of register ↔ memory
  - Or an atomic pair of instructions

Synchronization in RISC-V

- Load reserved: `lr.d rd,(rs1)`
  - Load from address in rs1 to rd
  - Place reservation on memory address
- Store conditional: `sc.d rd,rs2,(rs1)`
  - Store from rs2 to address in rs1
  - Succeeds if location not changed since the `lr.d`
    - Returns 0 in rd
  - Fails if location is changed
    - Returns non-zero value in rd

Synchronization in RISC-V

- Example 1: atomic swap (to test/set lock variable)

```
again:  lr.d x10,(x20)
        sc.d x11,x23,(x20) // X11 = status
        bne  x11,x0,again  // branch if store failed
        addi x23,x10,0     // X23 = loaded value
```

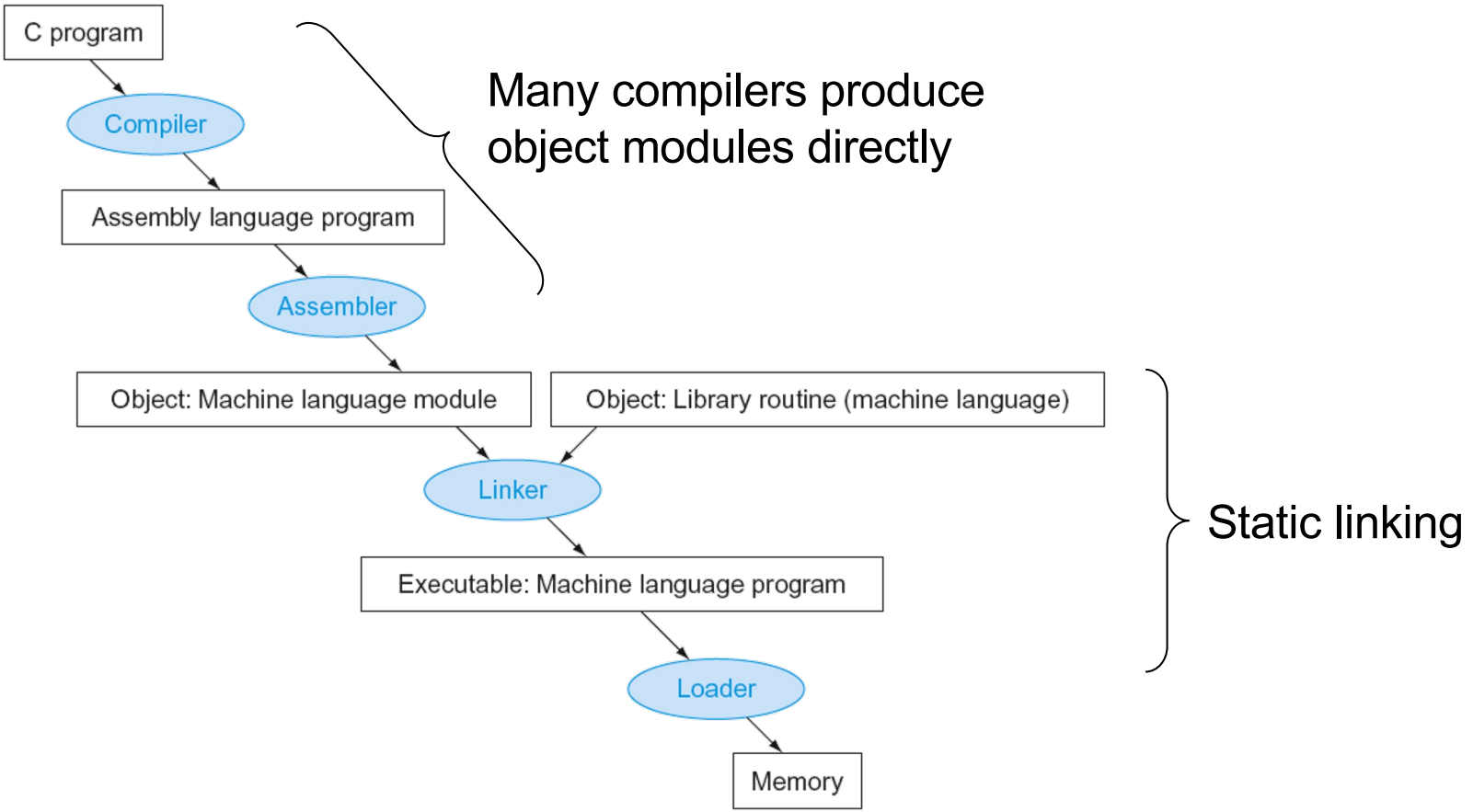
- Example 2: lock

```
        addi x12,x0,1          // copy locked value
again:  lr.d x10,(x20)         // read lock
        bne  x10,x0,again      // check if it is 0 yet
        sc.d x11,x12,(x20)     // attempt to store
        bne  x11,x0,again      // branch if fails
```

- Unlock:

```
        sd   x0,0(x20)         // free lock
```

Translation and Startup



Producing an Object Module

- Assembler (or compiler) translates program into machine instructions
- Provides information for building a complete program from the pieces
  - Header: described contents of object module
  - Text segment: translated instructions
  - Static data segment: data allocated for the life of the program
  - Relocation info: for contents that depend on absolute location of loaded program
  - Symbol table: global definitions and external refs
  - Debug info: for associating with source code

Linking Object Modules

- Produces an executable image
  1. Merges segments
  2. Resolve labels (determine their addresses)
  3. Patch location-dependent and external refs
- Could leave location dependencies for fixing by a relocating loader
  - But with virtual memory, no need to do this
  - Program can be loaded into absolute location in virtual memory space

Loading a Program

- Load from image file on disk into memory
  1. Read header to determine segment sizes
  2. Create virtual address space
  3. Copy text and initialized data into memory
     - Or set page table entries so they can be faulted in
  4. Set up arguments on stack
  5. Initialize registers (including sp, fp, gp)
  6. Jump to startup routine
     - Copies arguments to x10, … and calls main
     - When main returns, do exit syscall

Dynamic Linking

- Only link/load library procedure when it is called
  - Requires procedure code to be relocatable
  - Avoids image bloat caused by static linking of all (transitively) referenced libraries
  - Automatically picks up new library versions

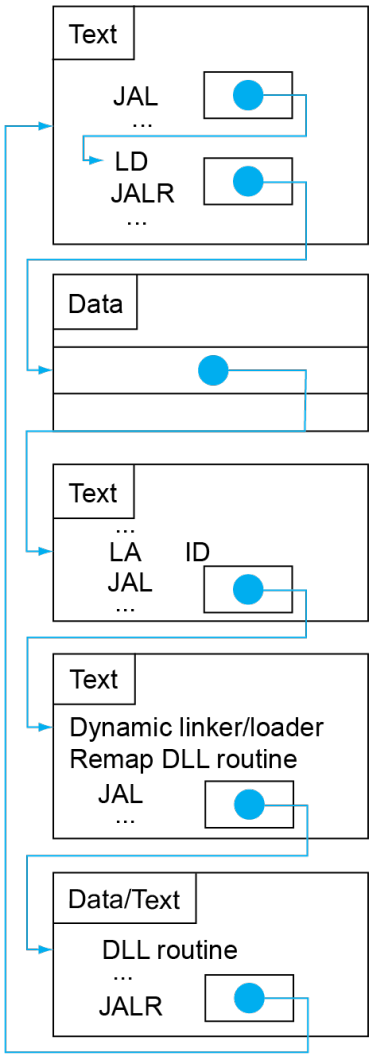
Lazy Linkage

Indirection table

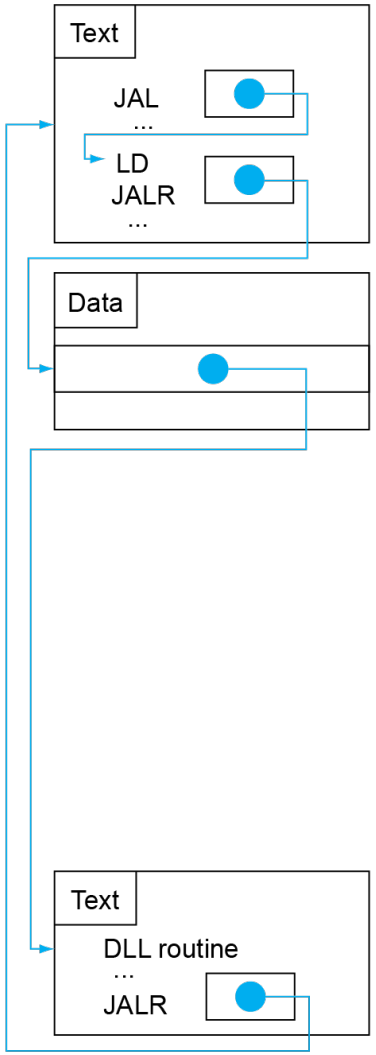
Stub: Loads routine ID,
Jump to linker/loader

Linker/loader code

Dynamically
mapped code



(a) First call to DLL routine



(b) Subsequent calls to DLL routine

C Sort Example

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
void swap(long long int v[],
          long long int k)
{
  long long int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

  - v in x10, k in x11, temp in x5

The Procedure Swap

```
swap:
  slli x6,x11,3     // reg x6 = k * 8
  add  x6,x10,x6    // reg x6 = v + (k * 8)
  ld   x5,0(x6)     // reg x5 (temp) = v[k]
  ld   x7,8(x6)     // reg x7 = v[k + 1]
  sd   x7,0(x6)     // v[k] = reg x7
  sd   x5,8(x6)     // v[k+1] = reg x5 (temp)
  jalr x0,0(x1)     // return to calling routine
```

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long long int v[], long long
int n)
{
  long long int i, j;
  for (i = 0; i < n; i += 1) {
    for (j = i - 1;
         j >= 0 && v[j] > v[j + 1];
         j -= 1) {
      swap(v,j);
    }
  }
}
```

  - v in x10, n in x11, i in x19, j in x20

The Outer Loop

- Skeleton of outer loop:
  - for (i = 0; i <n; i += 1) {

```
    li   x19,0              // i = 0
for1tst:
    bge  x19,x11,exit1   // go to exit1 if x19 ≥ x11 (i≥n)

    (body of outer for-loop)

    addi x19,x19,1       // i += 1
    j    for1tst         // branch to test of outer loop
exit1:
```

The Inner Loop

- Skeleton of inner loop:
  - for (j = i − 1; j >= 0 && v[j] > v[j + 1]; j − = 1) {

```
    addi x20,x19,-1     // j = i -1
for2tst:
    blt  x20,x0,exit2   // go to exit2 if X20 < 0 (j < 0)
    slli x5,x20,3       // reg x5 = j * 8
    add  x5,x10,x5      // reg x5 = v + (j * 8)
    ld   x6,0(x5)       // reg x6 = v[j]
    ld   x7,8(x5)       // reg x7 = v[j + 1]
    ble  x6,x7,exit2    // go to exit2 if x6 ≤ x7
    mv   x21, x10       // copy parameter x10 into x21
    mv   x22, x11       // copy parameter x11 into x22
    mv   x10, x21       // first swap parameter is v
    mv   x11, x20       // second swap parameter is j
    jal  x1,swap        // call swap
    addi x20,x20,-1     // j -= 1
    j    for2tst        // branch to test of inner loop
exit2:
```

Preserving Registers

- Preserve saved registers:

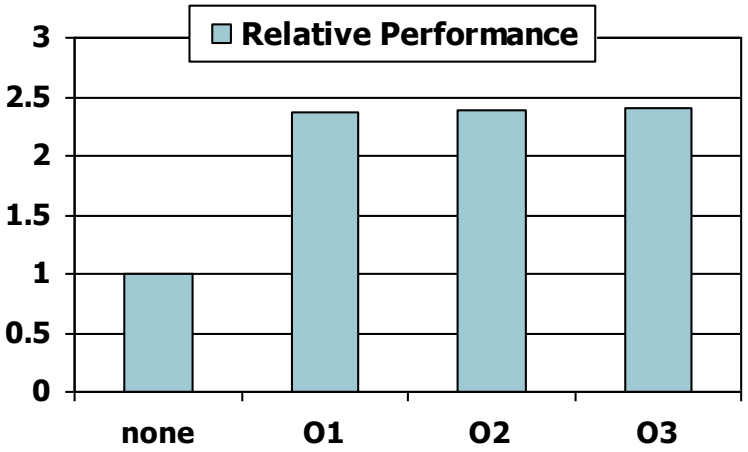
```
addi sp,sp,-40  // make room on stack for 5 regs
sd   x1,32(sp)  // save x1 on stack
sd   x22,24(sp) // save x22 on stack
sd   x21,16(sp) // save x21 on stack
sd   x20,8(sp)  // save x20 on stack
sd   x19,0(sp)  // save x19 on stack
```

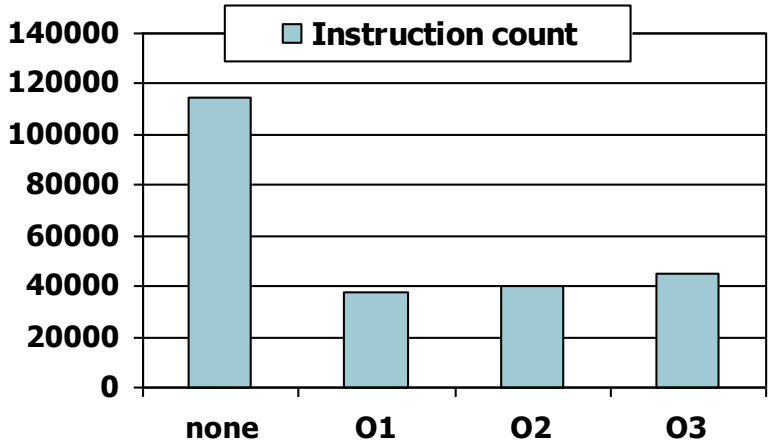
- Restore saved registers:

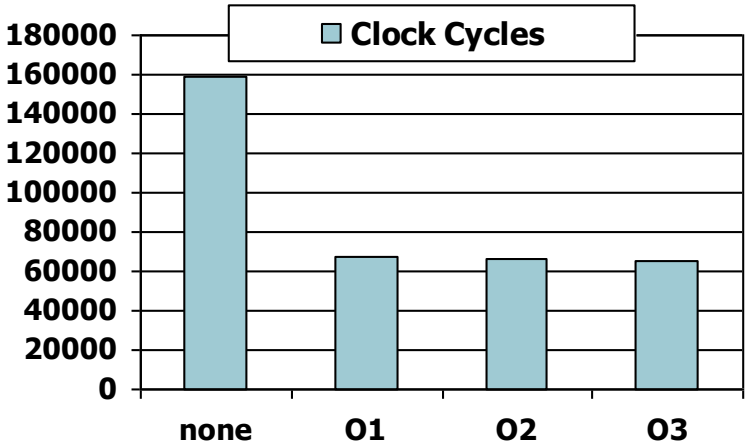
```
exit1:
  ld   x19,0(sp)  // restore x19 from stack
  ld   x20,8(sp)  // restore x20 from stack
  ld   x21,16(sp) // restore x21 from stack
  ld   x22,24(sp) // restore x22 from stack
  ld   x1,32(sp)  // restore x1 from stack
  addi sp,sp, 40  // restore stack pointer
  jalr x0,0(x1)
```

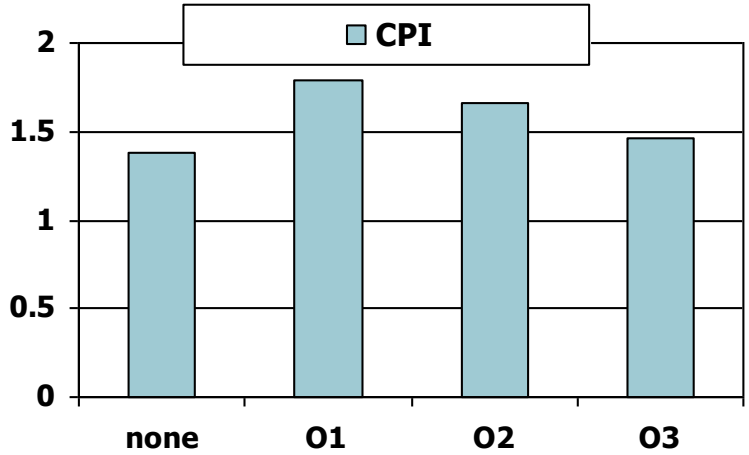
Effect of Compiler Optimization

Compiled with gcc for Pentium 4 under Linux

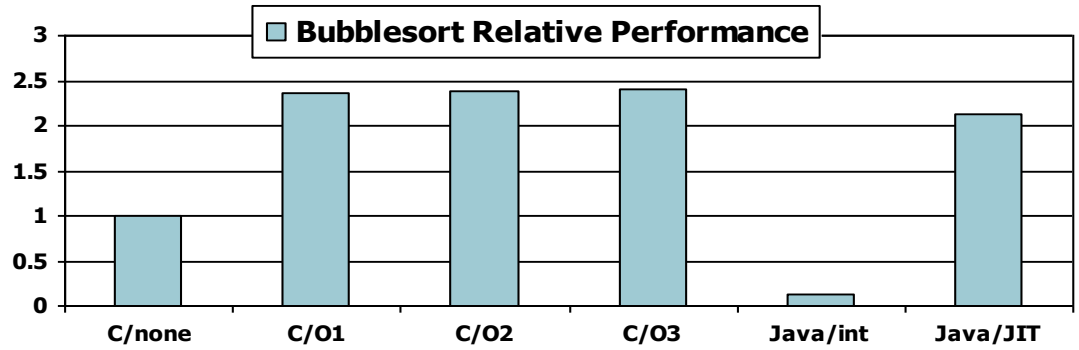


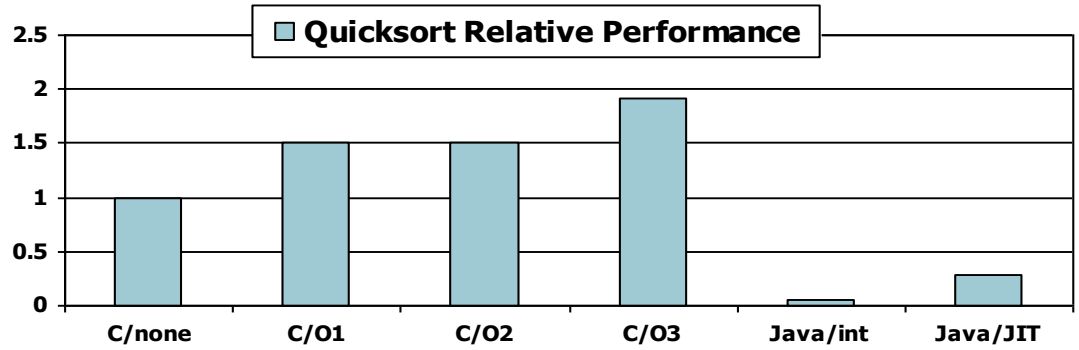


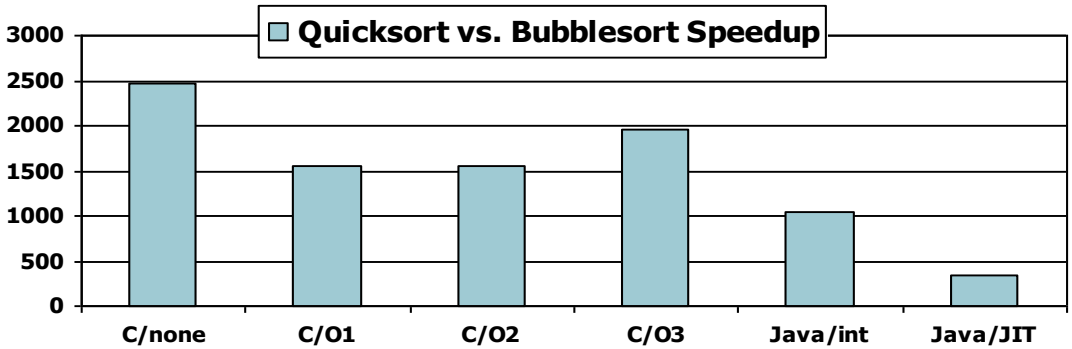




Effect of Language and Algorithm







Lessons Learnt

- Instruction count and CPI are not good performance indicators in isolation
- Compiler optimizations are sensitive to the algorithm
- Java/JIT compiled code is significantly faster than JVM interpreted
  - Comparable to optimized C in some cases
- Nothing can fix a dumb algorithm!

Arrays vs. Pointers

- Array indexing involves
  - Multiplying index by element size
  - Adding to array base address
- Pointers correspond directly to memory addresses
  - Can avoid indexing complexity

Example: Clearing an Array

```
clear1(int array[], int size) {
  int i;
  for (i = 0; i < size; i += 1)
    array[i] = 0;
}
```

```
clear2(int *array, int size) {
  int *p;
  for (p = &array[0]; p < &array[size];
       p = p + 1)
    *p = 0;
}
```

```
   li   x5,0       // i = 0
loop1:
   slli x6,x5,3    // x6 = i * 8
   add  x7,x10,x6  // x7 = address
                   // of array[i]
   sd   x0,0(x7)   // array[i] = 0
   addi x5,x5,1    // i = i + 1
   blt  x5,x11,loop1  // if (i<size)
                      // go to loop1
```

```
   mv x5,x10       // p = address
                   // of array[0]
   slli x6,x11,3   // x6 = size * 8
   add x7,x10,x6   // x7 = address
                   // of array[size]
loop2:
   sd x0,0(x5)     // Memory[p] = 0
   addi x5,x5,8    // p = p + 8
   bltu x5,x7,loop2
                   // if (p<&array[size])
                   // go to loop2
```

Comparison of Array vs. Ptr

- Multiply “strength reduced” to shift
- Array version requires shift to be inside loop
  - Part of index calculation for incremented i
  - c.f. incrementing pointer
- Compiler can achieve same effect as manual use of pointers
  - Induction variable elimination
  - Better to make program clearer and safer

Other RISC-V Instructions

- Base integer instructions (RV64I)
  - Those previously described, plus
  - slt, sltu, slti, sltui: set less than (like MIPS)
  - addw, subw, addiw: 32-bit add/sub
  - sllw, srlw, srlw, slliw, srliw, sraiw: 32-bit shift
- 32-bit variant: RV32I
  - registers are 32-bits wide, 32-bit operations

Instruction Set Extensions

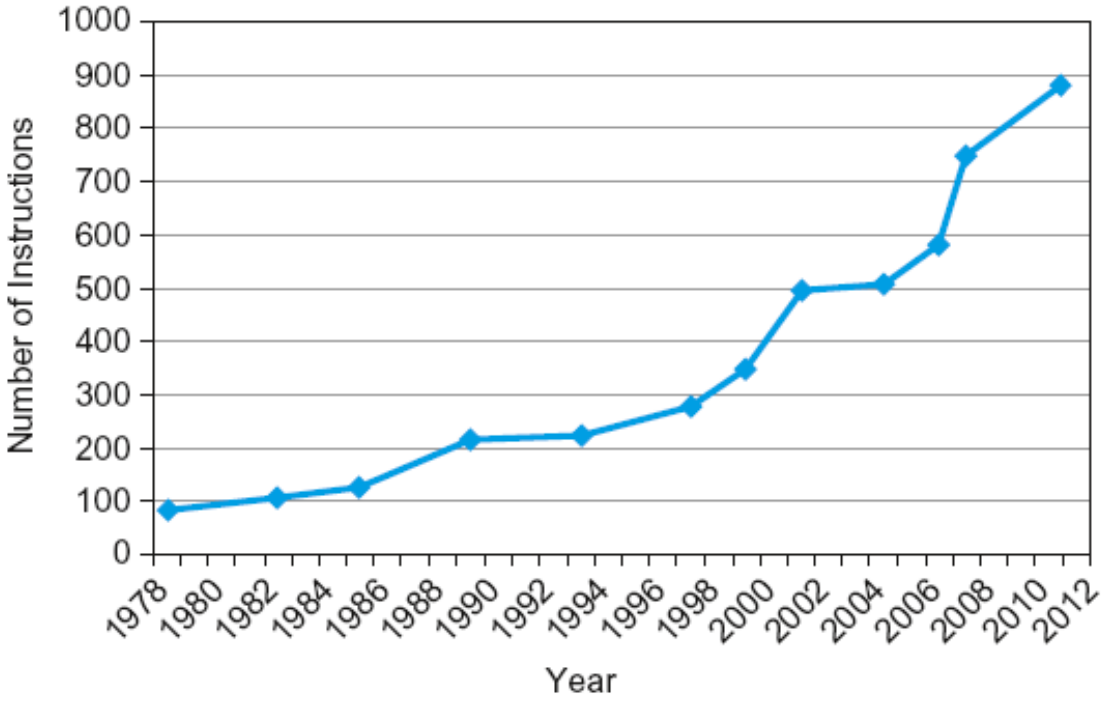
- M: integer multiply, divide, remainder
- A: atomic memory operations
- F: single-precision floating point
- D: double-precision floating point
- C: compressed instructions
  - 16-bit encoding for frequently used instructions

Fallacies

- Powerful instruction ⇒ higher performance
  - Fewer instructions required
  - But complex instructions are hard to implement
    - May slow down all instructions, including simple ones
  - Compilers are good at making fast code from simple instructions
- Use assembly code for high performance
  - But modern compilers are better at dealing with modern processors
  - More lines of code ⇒ more errors and less productivity

Fallacies

- Backward compatibility ⇒ instruction set doesn't change
  - But they do accrete more instructions



x86 instruction set

Pitfalls

- Sequential words are not at sequential addresses
  - Increment by 4, not by 1!
- Keeping a pointer to an automatic variable after procedure returns
  - e.g., passing pointer back via an argument
  - Pointer becomes invalid when stack popped

Concluding Remarks

- Design principles
  1. Simplicity favors regularity
  2. Smaller is faster
  3. Good design demands good compromises
- Make the common case fast
- Layers of software/hardware
  - Compiler, assembler, hardware
- RISC-V: typical of RISC ISAs